VajraKavach

Attack Recovery Report

Recovering After an Attack or Failure

Report Date:	April 7, 2026
Classification:	CONFIDENTIAL
Prepared By:	VajraKavach Security Team
Platform:	Secure Job Search & Professional Networking
Target:	`https://192.168.3.44`
Course:	CSE 345/545, IIIT-Delhi

Contents

1 Executive Summary **3**

2 Automated Service Restoration **3**
2.1 Systemd Auto-Restart . . . 3
2.2 Systemd Watchdog . . . 4
2.3 Health Check Timer . . . 5
2.4 Kernel Panic Auto-Reboot . . . 6
2.5 Service Dependency Chain . . . 6

3 Security State Persistence **7**
3.1 Token Revocation Persistence . . . 7
3.2 TOTP Lockout State Persistence . . . 7

4 Forensic Evidence & Integrity **8**
4.1 SHA-256 Hash-Chained Audit Logs . . . 8
4.2 Blockchain Integrity Layer . . . 8
4.3 File Integrity Verification . . . 9

5 Recovery Architecture Diagram **9**

6 Recovery Summary **9**
6.1 Recovery Guarantee . . . 10

1 Executive Summary

Attack recovery encompasses all mechanisms that enable a system to **restore service after an attack succeeds** or a failure occurs. This includes automatic service restart after crashes, detection of hanging processes, scheduled health monitoring, forensic logging for post-incident analysis, and cryptographic integrity verification of data.

VajraKavach implements **10 distinct recovery mechanisms** organized into three categories: automated service restoration, security state persistence, and forensic evidence.

2 Automated Service Restoration

2.1 Systemd Auto-Restart

Mechanism: `Restart=always` with `RestartSec=3` in systemd service unit.
Location: `/etc/systemd/system/vajrakavach-backend.service`
Configuration:

```
[Service]
Restart=always
RestartSec=3
StartLimitIntervalSec=60
StartLimitBurst=10
```

Failure Scenarios Covered:

- Process crash (segfault, unhandled exception, OOM kill)
- Intentional termination (SIGTERM, SIGKILL)
- Exit with non-zero code

Recovery Timeline:

Event	Time
Process crashes	$T = 0$
systemd detects exit	$T + 0$s
Restart delay	$T + 3$s
Uvicorn starts, lifespan begins	$T + 3.5$s
Database init, state reload	$T + 4$s
`READY=1` sent, serving requests	$T + 4.5$s

Recovery time: ∼4.5 seconds from crash to serving requests.
Restart throttling: Up to 10 restarts are allowed within a 60-second window. If the service crashes more than 10 times in 60 seconds (indicating a persistent failure), systemd stops attempting restarts to prevent crash loops. This can be reset with `systemctl reset-failed`.

2.2 Systemd Watchdog

Mechanism: `WatchdogSec=30` with `Type=notify` — process must send periodic heartbeats.
Location: Service: `/etc/systemd/system/vajrakavach-backend.service`. Application: `backend/app/main.py`
How It Works:

1. The service is configured with `Type=notify` and `WatchdogSec=30`
2. On startup, the FastAPI application sends `READY=1` to systemd via the `sd_notify` protocol (Unix datagram socket)
3. An async background task sends `WATCHDOG=1` every 15 seconds (half the watchdog interval)
4. If systemd does not receive a heartbeat within 30 seconds, it considers the process **hung** and kills it with SIGABRT
5. The `Restart=always` policy then restarts the service

Failure Scenarios Covered (beyond simple crashes):

- **Deadlock:** Event loop stuck in a deadlocked state — heartbeat stops
- **Infinite loop:** CPU-bound infinite loop blocks the async event loop — heartbeat stops
- **Resource exhaustion:** Memory thrashing or I/O blocking prevents async tasks from running
- **Database connection hang:** If all DB connections are stuck, the event loop may still run but the app is effectively dead

Implementation:

```
def _sd_notify(state: str):
    addr = os.environ.get("NOTIFY_SOCKET")
    if not addr:
        return
    sock = socket.socket(socket.AF_UNIX, socket.SOCK_DGRAM)
    sock.connect(addr)
    sock.sendall(state.encode())
    sock.close()

async def _watchdog_loop():
    interval = int(os.environ.get("WATCHDOG_USEC", "0")) / 1_000_000 / 2
    while True:
        _sd_notify("WATCHDOG=1")
        await asyncio.sleep(interval)
```

Recovery time: ≤30s (watchdog timeout) + 4.5s (restart) = ≤34.5 seconds.

2.3 Health Check Timer

Mechanism: Systemd timer executes a health check script every 2 minutes.
Location:

- Script: `/usr/local/bin/vajrakavach-healthcheck.sh`
- Timer: `/etc/systemd/system/vajrakavach-healthcheck.timer`
- Service: `/etc/systemd/system/vajrakavach-healthcheck.service`

How It Works:

1. Every 2 minutes, systemd triggers the health check service
2. The script sends an HTTPS request to `https://192.168.3.44/api/health`
3. If the request fails, it retries up to 3 times with 5-second delays
4. If all 3 retries fail, it logs an alert to syslog and restarts the backend service

Failure Scenarios Covered (beyond watchdog):

- **Backend serving but Nginx down:** Health check via HTTPS goes through Nginx — catches Nginx failures too
- **Backend alive but not serving:** Process is running and sending WATCHDOG=1 but the HTTP handler is broken (e.g., middleware exception, route registration failure)
- **TLS certificate expiry:** HTTPS health check would fail if the certificate is expired or invalid
- **Network connectivity issues:** Detects if the service is unreachable from the network perspective

Note: This is implemented as a **systemd timer**, not a cron job. Systemd timers provide better integration with service management (dependencies, logging via journalctl, status visibility via `systemctl list-timers`).
Recovery time: $\leq$2 minutes (timer interval) + 15s (3 retries $\times$ 5s) + 4.5s (restart) $\approx$ 2.5 minutes worst case.

2.4 Kernel Panic Auto-Reboot

Mechanism: `kernel.panic=10` — automatic system reboot 10 seconds after a kernel panic.
Location: Runtime: `sysctl -w kernel.panic=10`. Persistent: `/etc/sysctl.d/99-vajrakavach-panic.conf`
Failure Scenarios Covered:

- **Kernel panic:** Critical kernel error (hardware failure, driver bug, memory corruption)
- **OOM with panic:** If `vm.panic_on_oom=1` is set, OOM conditions trigger a panic and reboot

Recovery Timeline:

Event	Time
Kernel panic occurs	$T = 0$
System reboots	$T + 10$s
BIOS/UEFI POST	$T + 30$s (approx.)
OS boots, systemd starts services	$T + 60$s (approx.)
VajraKavach backend ready	$T + 65$s (approx.)

Recovery time: ∼1 minute from kernel panic to service restoration.
Without this setting: The default `kernel.panic=0` halts the system permanently after a kernel panic, requiring manual intervention (physical access or remote IPMI/iLO) to reboot.

2.5 Service Dependency Chain

Mechanism: `After=postgresql.service` and `Requires=postgresql.service` in systemd unit.
Location: `/etc/systemd/system/vajrakavach-backend.service`
Purpose: After a system reboot (including kernel panic auto-reboot), services must start in the correct order. This ensures PostgreSQL is running before the backend attempts database connections.
Without dependency chain: The backend would start before PostgreSQL, fail to connect to the database, and crash. The `Restart=always` policy would restart it, but it would take several crash-restart cycles before PostgreSQL is ready.

3 Security State Persistence

3.1 Token Revocation Persistence

Mechanism: JWT revocation blocklist persisted to PostgreSQL, reloaded on startup.
Location: `backend/app/services/auth_service.py`
How It Works:

1. In-memory set provides O(1) revocation checks during normal operation
2. Pending revocations are batched and flushed to the `revoked_tokens` table
3. On shutdown, all pending revocations are flushed via `flush_pending_revocations()`
4. On startup, `load_revoked_cache()` loads all non-expired revoked tokens from the database
5. Expired tokens are automatically cleaned up via `cleanup_expired_tokens()`

Purpose: If an attacker steals a token and the administrator revokes it, the revocation must survive server restarts. Without persistence, a restart would "un-revoke" the stolen token, giving the attacker access again.

3.2 TOTP Lockout State Persistence

Mechanism: TOTP lockout state (attempts, lockout level, locked-until) persisted to PostgreSQL.
Location: `backend/app/services/totp_service.py`
How It Works:

1. In-memory dict (`_totp_attempts`) provides fast lockout checks
2. On shutdown, `sync_totp_lockout_to_db()` writes all state to the `totp_lockouts` table
3. On startup, `load_totp_lockout_from_db()` restores the state
4. The lockout level (escalation counter) persists across sessions, ensuring progressive lockout escalation is not reset by server restarts

Purpose: An attacker cannot defeat progressive lockout by causing the server to restart. The lockout level (60 min → 300 min → 1500 min) continues to escalate regardless of restarts.

4 Forensic Evidence & Integrity

4.1 SHA-256 Hash-Chained Audit Logs

Mechanism: Every audit log entry includes a SHA-256 hash of itself concatenated with the previous entry's hash.
Location: `backend/app/services/audit_service.py`
Hash computation:

```
payload = f"{previous_hash}|{action}|{user_id}|{resource}|
            {resource_id}|{details}|{ip_address}|{timestamp}"
hash = SHA256(payload)
```

Purpose: After an attack, investigators need to reconstruct the sequence of events. Hash chaining ensures that if an attacker modifies, deletes, or reorders any log entry, the chain breaks and tampering is detectable.
Logged fields: Action type, user ID, resource type, resource ID, IP address, timestamp, and free-form details.

4.2 Blockchain Integrity Layer

Mechanism: Merkle tree blocks with proof-of-work over audit log entries.
Location: `backend/app/services/blockchain_service.py`
Architecture:

1. Audit log entries are collected in batches of 10 (`BLOCK_SIZE=10`)
2. A Merkle root is computed from the SHA-256 hashes of all entries in the batch
3. A proof-of-work nonce is mined such that the block hash starts with 2 leading zeros (`DIFFICULTY=2`)
4. The block is chained to the previous block via its hash
5. The genesis block uses a predefined hash (`"0" * 64`)

Block structure:

```
Block Hash = SHA256(block_number | previous_hash | merkle_root |
                    timestamp | nonce)
```

Post-attack verification: To verify audit log integrity after an attack, the system can:

1. Recompute each block's Merkle root from its constituent entries
2. Verify the proof-of-work (hash starts with `DIFFICULTY` zeros)
3. Verify each block's `previous_hash` matches the previous block's hash
4. Any discrepancy indicates tampering

Purpose: Provides cryptographic proof that audit logs have not been modified post-attack. This is essential for forensic analysis, incident response, and regulatory compliance.

4.3 File Integrity Verification

Mechanism: Fernet encryption (AES-128-CBC + HMAC-SHA256) with SHA-256 checksums.
Location: `backend/app/services/encryption_service.py`
How It Works:

- Resume files are encrypted with Fernet before storage
- Fernet includes HMAC-SHA256 authentication — any tampering with the ciphertext is detected during decryption (`InvalidToken` exception)
- Additionally, SHA-256 checksums are stored per file for independent integrity verification

Post-attack recovery: If an attacker modifies encrypted files on disk, decryption will fail with an HMAC validation error, alerting the system to file tampering.

5 Recovery Architecture Diagram

The recovery mechanisms form a layered detection and response system:

Layer	Mechanism	Detection Time	Recovery Time
1	Systemd Auto-Restart	Immediate (on crash)	∼4.5 seconds
2	Systemd Watchdog	$\leq$ 30 seconds (on hang)	∼34.5 seconds
3	Health Check Timer	$\leq$ 2 minutes (on unresponsive)	∼2.5 minutes
4	Kernel Panic Reboot	10 seconds (on kernel panic)	∼1 minute

Cascading coverage:

- If the process **crashes**, Layer 1 handles it in 4.5 seconds.
- If the process **hangs** (no crash), Layer 2 detects it within 30 seconds.
- If the process is **alive but unresponsive** (watchdog still ticking but HTTP broken), Layer 3 catches it within 2 minutes.
- If the **entire OS fails** (kernel panic), Layer 4 reboots in 10 seconds.

No failure mode goes undetected for more than 2.5 minutes.

6 Recovery Summary

#	Recovery Mechanism	Category
1	Systemd auto-restart (Restart=always, 3s delay)	Service Restoration
2	Systemd watchdog (WatchdogSec=30, sd_notify)	Service Restoration
3	Health check timer (every 2 min, 3 retries)	Service Restoration
4	Kernel panic auto-reboot (10s delay)	Service Restoration
5	Service dependency chain (Requires=postgresql)	Service Restoration
6	Token revocation persistence (DB-backed)	State Persistence
7	TOTP lockout state persistence (DB-backed)	State Persistence
8	SHA-256 hash-chained audit logs	Forensic Evidence
9	Blockchain integrity layer (Merkle + PoW)	Forensic Evidence
10	File integrity verification (Fernet HMAC + SHA-256)	Forensic Evidence
Total Recovery Mechanisms		**10**

6.1 Recovery Guarantee

Maximum Downtime Guarantee

- **Process crash:** $\leq$ 5 seconds recovery
- **Process hang:** $\leq$ 35 seconds recovery
- **Unresponsive service:** $\leq$ 2.5 minutes recovery
- **Kernel panic:** $\leq$ 1.5 minutes recovery
- **Security state (lockouts, revocations):** Survives all restarts — zero data loss
- **Audit trail integrity:** Cryptographically verifiable post-incident